Faculty of Engineering
Alexanderia University
Electrical Engineering Department
Communications and Electronics Engineering
EEC 243 Microprocessors and Interfacing

Mini Project I:
General signal generator

NAME	Section	ID
احمد باسم عبدالسلام ابوكيلة	1	24011105
محمد احمد فكيه طه	2	23010718
محمد ايمن رجب الصايغ	1	23010730
محمد محمود السيد محمد علي شطا	2	23010781

1. Introduction

- In this project, a general signal generator is designed and implemented using MATLAB. The program allows the user to create different types of signals such as **DC, ramp, exponential, and sinusoidal signals.**
- The user first enters basic parameters like the **sampling frequency, time range, and breakpoints**. Based on these inputs, the signal is generated as multiple regions, where each region can have a different signal type.
- After generating the signal, the program allows the user to apply operations such as **amplitude scaling, time shifting, time reversal, expansion, and compression**. The results are then displayed using plots in the time domain.
- This project helps in understanding how signals are created, modified, and analyzed in a simple and interactive way.

2.MATLAB Code Implementation

1. User Input

```
clear; clc; close all;
fprintf('=== General Signal Generator ===\n\n');

%% ========================
%% 1. GLOBAL PARAMETERS
%% ========================
fs = input('Enter sampling frequency (Hz): ');
while fs <= 0
    fs = input('Must be > 0! Enter sampling frequency: ');
end

t_start = input('Enter start time: ');
t_end   = input('Enter end time: ');
while t_start >= t_end
    t_start = input('Start must be < End! Enter start: ');
    t_end   = input('Enter end: ');
end

dt = 1/fs;
t_total = t_start:dt:t_end;

%% ========================
%% 2. BREAKPOINTS
%% ========================
num_bp = input('Enter number of breakpoints: ');
while num_bp < 0 || num_bp ~= floor(num_bp)
    num_bp = input('Must be non-negative integer! Enter again: ');
end

breakpoints = [];
if num_bp > 0
    fprintf('Enter breakpoint positions:\n');
    for i = 1:num_bp
        bp = input(sprintf('Breakpoint %d: ', i));
        while bp <= t_start || bp >= t_end
            bp = input('Must be inside range! Enter again: ');
        end
        breakpoints = [breakpoints bp];
    end
    breakpoints = unique(sort(breakpoints));
end

all_breaks = [t_start breakpoints t_end];
num_regions = length(all_breaks) - 1;
```

1.1 Overview

- The first stage of the program is responsible for collecting all necessary input parameters from the user in order to construct the signal. These inputs define the time axis, sampling resolution, and how the signal is segmented into different regions.
- The program ensures that all entered values are valid before proceeding, which improves robustness and prevents runtime errors.

1.2 Input Parameters

The user is asked to provide the following inputs:

1. Sampling Frequency (Fs)

The sampling frequency determines how many samples are taken per second. It directly affects the resolution of the generated signal.

```
fs = input('Enter sampling frequency (Hz): ');

while fs <= 0

   fs = input('Must be > 0! Enter sampling frequency: ');

end
```

- The program enforces that the sampling frequency must be greater than zero.
- This is important because a zero or negative sampling rate is physically meaningless.

---

2. Time Range

The user specifies the start and end of the time interval:

```
t_start = input('Enter start time: ');

t_end   = input('Enter end time: ');

while t_start >= t_end

   t_start = input('Start must be < End! Enter start: ');

   t_end   = input('Enter end: ');
```

```
end
```

- The program ensures that the start time is strictly less than the end time.
- This guarantees a valid time axis.

---

3. Time Vector Construction

```
dt = 1/fs;

t_total = t_start:dt:t_end;
```

- The time step (dt) is computed as the inverse of the sampling frequency.
- A uniformly spaced time vector is generated, which will be used to build the signal.

---

4. Breakpoints

Breakpoints define where the signal changes its behavior.

```
num_bp = input('Enter number of breakpoints: ');

while num_bp < 0 || num_bp ~= floor(num_bp)

   num_bp = input('Must be non-negative integer! Enter again: ');

end
```

- The number of breakpoints must be a non-negative integer.
- Each breakpoint represents a boundary between two signal regions.

---

5. Breakpoint Positions

```
breakpoints = [];
for i = 1:num_bp
   bp = input(sprintf('Breakpoint %d: ', i));
   while bp <= t_start || bp >= t_end
      bp = input('Must be inside range! Enter again: ');
   end
   breakpoints = [breakpoints bp];
end
breakpoints = unique(sort(breakpoints));
```

- Each breakpoint must lie between start and end.

The program:

- Remove duplicates

- Sorts the breakpoint

6. Region Definition

```
all_breaks = [t_start breakpoints t_end];
num_regions = length(all_breaks) - 1;
```

- The full-time interval is divided into multiple regions.
- Each region will have its own signal type.

1.3 Example of User Input

```
=== General Signal Generator ===

Enter sampling frequency (Hz): 100
Enter start time: -2
Enter end time: 2
Enter number of breakpoints: 1
Enter breakpoint positions:
Breakpoint 1: 0
```

1.4 Error Handling Strategy

Input Type	Validation Rule	Purpose
Sampling Frequency	Fs>0	Prevent invalid time step
Time Range	Start < end	Ensure valid interval
Breakpoints Count	Integer ≥ 0	Logical segmentation
Breakpoint Position	between start and end.	Avoid out-of-range errors

1.5 Summary

The user input stage builds the foundation of the entire system by:

- Defining the time axis
- Dividing the signal into regions
- Ensuring all inputs are valid

2. Signal Generation

```
%% ========================
```

```
%% 3. GENERATE SIGNAL
%% ========================

y_total = zeros(size(t_total));

for r = 1:num_regions

    fprintf('\n--- Region %d [%.2f -> %.2f] ---\n', ...
        r, all_breaks(r), all_breaks(r+1));

    % region time
    if r < num_regions
        idx = t_total >= all_breaks(r) & t_total < all_breaks(r+1);
    else
        idx = t_total >= all_breaks(r) & t_total <= all_breaks(r+1);
    end

    t_region = t_total(idx);

    %% choose signal
    disp('1=DC  2=Ramp  3=Polynomial  4=Exponential  5=Sinusoidal');
    type = input('Enter choice: ');

    while type < 1 || type > 5
        type = input('Invalid! Enter (1-5): ');
    end

    %% generate region signal
    switch type

        case 1 % DC
            A = input('Amplitude: ');
            y_region = A * ones(size(t_region));

        case 2 % Ramp
            m = input('Slope: ');
            b = input('Intercept: ');
            y_region = m*t_region + b;

        case 3 % Polynomial
            a = input('Amplitude: ');
            n = input('Power: ');
            b = input('Intercept: ');
            y_region = a*(t_region.^n) + b;

        case 4 % Exponential
            A = input('Amplitude: ');
            k = input('Exponent: ');
            y_region = A * exp(k*t_region);

        case 5 % Sinusoidal
            A = input('Amplitude: ');
            f = input('Frequency: ');
            phi = input('Phase: ');
            y_region = A * sin(2*pi*f*t_region + phi);
```

```
    end

    % assign to full signal
    y_total(idx) = y_region;
end

%% ========================
%% 4. ORIGINAL PLOT
%% ========================
figure;
plot(t_total, y_total, 'LineWidth', 2);
grid on;
title('Original Signal');
xlabel('Time');
ylabel('Amplitude');
```

2.1 Overview

In this section, the program generates the signal based on the user inputs. The signal is constructed as a piecewise function, meaning it is divided into multiple regions, and each region can have a different signal type.

The number of regions depends on the number of breakpoints entered by the user.

2.2 Signal Initialization

y_total = zeros(size(t_total));

- A zero vector is created to store the final signal.
- It has the same size as the time vector.
- Each region will be generated separately and then stored in this vector.

2.3 Dividing the Signal into Regions

for r = 1:num_regions

- The program loops over each region.
- Each region is defined between two consecutive breakpoints.

2.4 Extracting Region Time

```
if r < num_regions
   idx = t_total >= all_breaks(r) & t_total < all_breaks(r+1);
else
   idx = t_total >= all_breaks(r) & t_total <= all_breaks(r+1);
end

t_region = t_total(idx);
```

- The program selects the time samples that belong to the current region.
- idx is a logical index used to extract only the required portion of time.
- This ensures that each region is processed independently.

2.5 Selecting Signal Type

```
disp('1=DC  2=Ramp  3=Polynomial  4=Exponential  5=Sinusoidal');

type = input('Enter choice: ');
```

- The user selects the type of signal for each region.
- The program supports five types of signals.

2.6 Generating Signal in Each Region

The signal is generated based on the selected type:

1. DC Signal

```
y_region = A * ones(size(t_region));
```

- Produces a constant signal.

---

2. Ramp Signal

```
y_region = m*t_region + b;
```

- Produces a linear signal.

---

3. Polynomial Signal

```
y_region = a*(t_region.^n) + b;
```

- Produces a polynomial signal of order (n).

---

4. Exponential Signal

```
y_region = A * exp(k*t_region);
```

- Produces an exponential signal.

5. Sinusoidal Signal

```
y_region = A * sin(2*pi*f*t_region + phi);
```

- Produces a sinusoidal (wave) signal.

2.7 Combining All Regions

```
y_total(idx) = y_region;
```

- Each generated region is placed into its correct position in the full signal.
- After the loop finishes, y_total contains the complete signal.

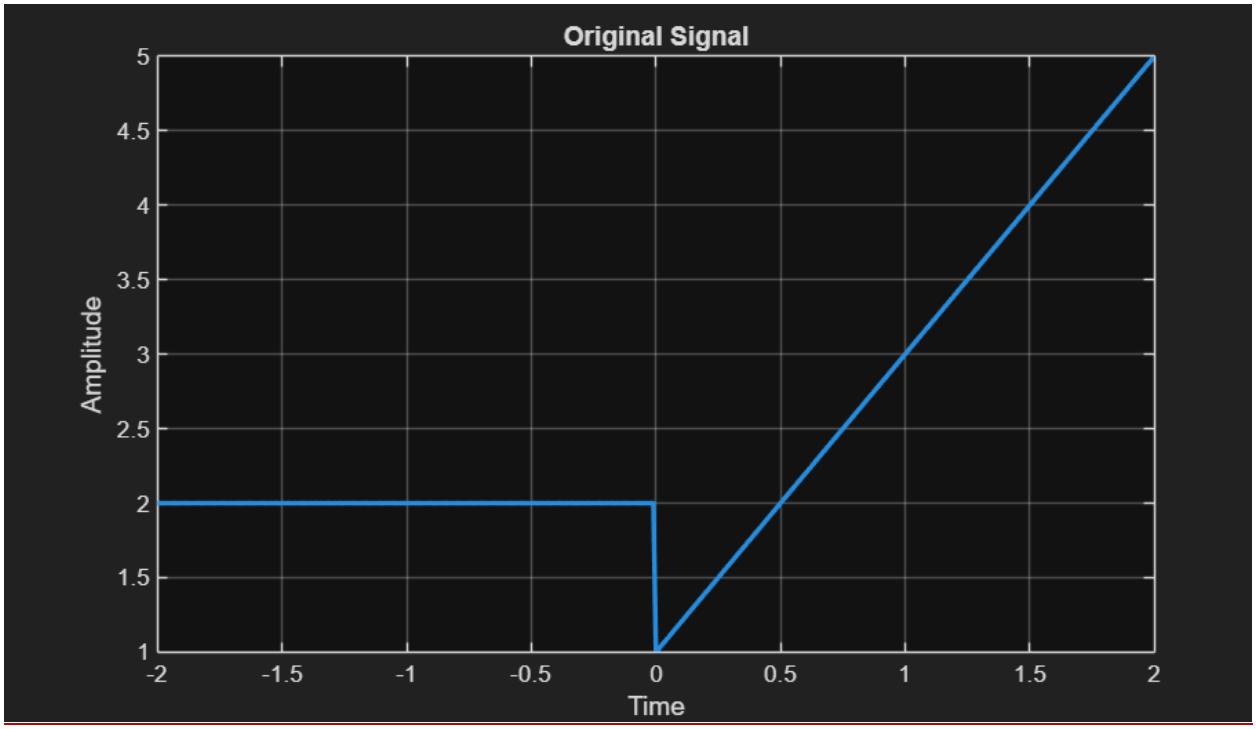
2.8 Plotting the Signal

```
figure;

plot(t_total, y_total, 'LineWidth', 2);

grid on;

title('Original Signal');

xlabel('Time');

ylabel('Amplitude');
```

- The final signal is plotted in the time domain.
- This allows visualization of how the signal changes across regions.

2.9 Example

```
--- Region 1 [-2.00 -> 0.00] ---
1=DC  2=Ramp  3=Polynomial  4=Exponential  5=Sinusoidal
Enter choice: 1
Amplitude: 4

--- Region 2 [0.00 -> 2.00] ---
1=DC  2=Ramp  3=Polynomial  4=Exponential  5=Sinusoidal
Enter choice: 3
Amplitude: 2
Power: 2
Intercept: 1
```



2.10 Summary

The signal generation process:

- Divides the time into regions
- Generates a specific signal in each region
- Combines all regions into one signal
- Displays the final result

3.signal operations

```
%% ========================
%% 5. OPERATIONS
%% ========================
% Start with the original generated signal
y_new = y_total;
t_new = t_total;

while true
    disp('Choose Operation:');
    disp('1-Amplitude Scaling');
    disp('2-Time Reversal');
    disp('3-Time Shift');
    disp('4-Expansion');
    disp('5-Compression');
    disp('6-None');
    op_id = input('Enter operation: ');

    if op_id == 1
        op_val = input('Enter scaling value: ');
    elseif op_id == 3
        op_val = input('Enter shift value: ');
    elseif op_id == 4
        op_val = input('Enter expansion factor: ');
    elseif op_id == 5
        op_val = input('Enter compression factor: ');
    else
        op_val = 0;
    end

    [y_new, t_new] = operation(y_new, t_new, op_id, op_val);

    %% ========================
    %% 6. SIGNAL PROPERTIES
    %% ========================
    tol = 1e-6;
    % 1. Symmetry Check (Even/Odd)
    t_min = t_new(1);
    t_max = t_new(end);
    is_even = false;
    is_odd = false;
    if abs(t_min + t_max) < tol
        y_flipped = flip(y_new);
        if all(abs(y_new - y_flipped) < tol)
            is_even = true;
        end
        if all(abs(y_new + y_flipped) < tol)
            is_odd = true;
        end
    end
    %% -------- Causality --------
    is_causal = all(abs(y_new(t_new < 0)) < tol);
    %% -------- PRINT --------
```

```
    fprintf('\nSignal Properties: ');
    if is_even
        fprintf('Even, ');
    elseif is_odd
        fprintf('Odd, ');
    else
        fprintf('Neither Even nor Odd, ');
    end
    if is_causal
        fprintf('Causal, ');
    else
        fprintf('Non-Causal, ');
    end

    %% ========================
    %% 7. ENERGY CALCULATION
    %% ========================
    dt = diff(t_new);
    E = sum(abs(y_new(1:end-1)).^2 .* dt);
    fprintf('\nSignal Energy = %.6f\n', E);

    %% Ask for another operation
    another = input('\nDo you want to perform another operation? (1 = Yes, 0 = No): ');
    if another == 0
        break;  % exit the loop
    end
end

%% ========================
%% 7. OPERATIONS FUNCTION
%% ========================
function [sig_out, t_out] = operation(sig_in, t_in, op_id, op_val)
switch op_id
case 1 % Amplitude Scaling
        sig_out = sig_in * op_val;
        t_out = t_in;
case 2 % Time Reversal
        sig_out = flip(sig_in);
        t_out = -flip(t_in);
case 3 % Time Shift
        t_out = t_in + op_val;
        sig_out = sig_in;
case 4 % Expansion
        t_out = linspace(t_in(1), t_in(end), length(sig_in)*op_val);
        sig_out = interp1(t_in, sig_in, t_out, 'linear', 0);
case 5 % Compression
        t_out = linspace(t_in(1), t_in(end), floor(length(sig_in)/op_val));
        sig_out = interp1(t_in, sig_in, t_out, 'linear', 0);
case 6 % None
        sig_out = sig_in;
        t_out = t_in;
otherwise
        error('Invalid operation');
end
%% plotting
```

```
figure;
subplot(2,1,1);
plot(t_in, sig_in, 'LineWidth',2);
title('Input Signal'); grid on;
subplot(2,1,2);
plot(t_out, sig_out, 'LineWidth',2);
title('Output Signal'); grid on;
end
```

3.1 Overview

After generating the original signal, the program allows the user to apply different operations on the signal. These operations modify either the amplitude or the time axis of the signal.

3.2 Available Operations

The user can choose one of the following operations:

```
disp('Choose Operation:');
disp('1-Amplitude Scaling');
disp('2-Time Reversal');
disp('3-Time Shift');
disp('4-Expansion');
disp('5-Compression');
disp('6-None');
```

3.3 Operation Selection

```
op_id = input('Enter operation: ');
```

- The user selects the desired operation by entering a number from 1 to 6.

3.4 Loop Control and User Input

```
    another = input('\nDo you want to perform another operation? (1 = Yes, 0 = No): ');
    if another == 0
        break;  % exit the loop
    end
```

- This code prompts the user to continue or stop the program after each operation.
- If the user enters 0, the break statement exits the loop; otherwise, execution continues.

3.5 Applying the Operation

```
[y_new, t_new] = operation(y_total, t_total, op_id, op_val);
```

- The operation is applied using a separate function.
- The output is a new signal and possibly a modified time vector

3.6 Operation Implementation

1. Amplitude Scaling

```
sig_out = sig_in * op_val;
```

- Multiplies the signal amplitude by a constant.

2. Time Reversal

```
sig_out = flip(sig_in);
t_out = -flip(t_in);
```

- Reflects the signal around the vertical axis.

3. Time Shift

```
t_out = t_in + op_val;

sig_out = sig_in;
```

- Shifts the signal in time.
- Uses interpolation to calculate new values.

4. Expansion

```
sig_out = interp1(t_in, sig_in, t_in / op_val, 'linear', 0);
```

- Stretches the signal in time (slower signal).

5. Compression

```
sig_out = interp1(t_in, sig_in, t_in * op_val, 'linear', 0);
```

- Compresses the signal (faster signal).

6. No Operation

```
sig_out = sig_in;

t_out = t_in;
```

3.7 Plotting Results

```
subplot(2,1,1);

plot(t_in, sig_in);

subplot(2,1,2);

plot(t_out, sig_out);
```

- The program displays:
  - Original signal
  - Modified signal

---

4. Signal Analysis

4.1 Overview

After applying operations, the program analyzes the resulting signal by checking its properties and calculating its energy.

4.2 Symmetry (Even / Odd)

```
% 1. Symmetry Check (Even/Odd)

t_min = t_new(1);
t_max = t_new(end);

is_even = false;
is_odd = false;

if abs(t_min + t_max) < tol

    y_flipped = flip(y_new);

    if all(abs(y_new - y_flipped) < tol)
        is_even = true;
    end

    if all(abs(y_new + y_flipped) < tol)
        is_odd = true;
    end
end
```

- The signal is compared with its flipped version:

4.3 Causality

```
is_causal = all(abs(y_new(t_new < 0)) < tol);
```

The program checks if all values before (t = 0) are zero.

4.4 Energy Calculation

```
%% ========================
%% 6. ENERGY CALCULATION
%% ========================
dt = diff(t_new);
E = sum(abs(y_new(1:end-1)).^2 .* dt);
fprintf('\nSignal Energy = %.6f\n', E);
```

- The program approximates the integral numerically.

4.5 Output Results

The program prints:

- Whether the signal is even or odd
- Whether it is causal
- The total energy

Examples

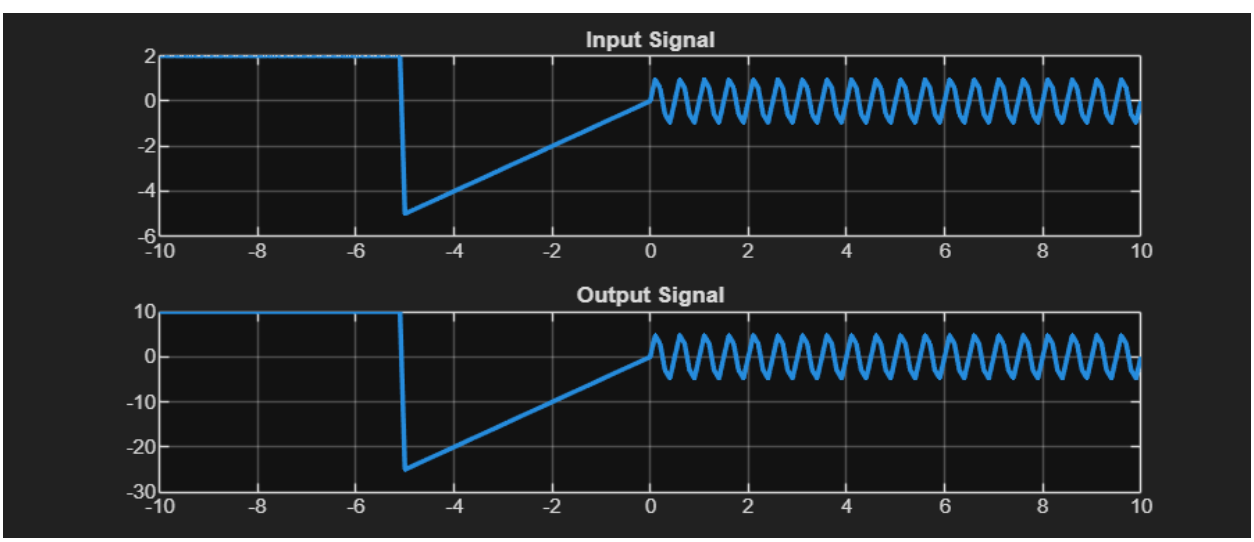


Figure 1: Amplitude Scaling (by 5)

```
Signal Properties: Neither Even nor Odd, Non-Causal,
Signal Energy = 1698.125000
```

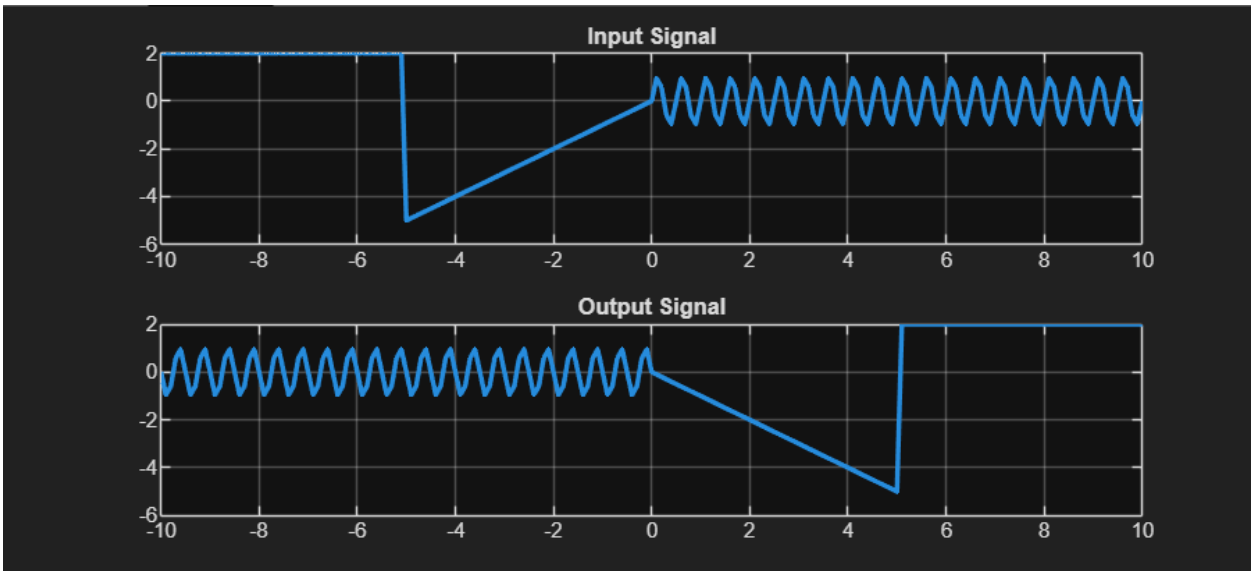


Figure 2: time reversal

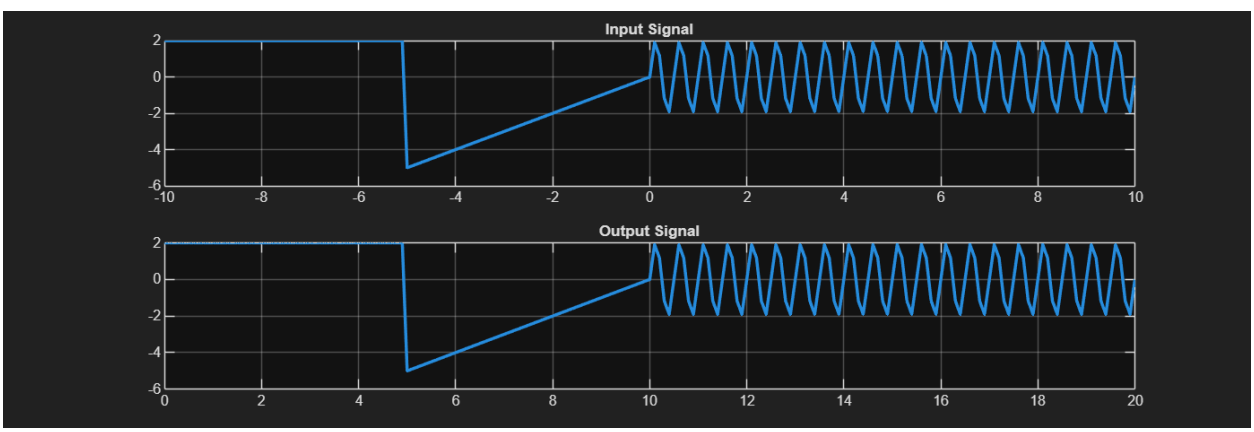


Figure 2: Time Shift (by 10)